

Auto-Dreamer: Learning Offline Memory Consolidation for Language Agents

Chongrui Ye^{*1} Yuxiang Liu^{*1} Yu Wang² Haofei Yu¹
Yining Zhao¹ Ge Liu¹ Julian McAuley² Jiaxuan You¹

¹University of Illinois Urbana-Champaign ²University of California San Diego

Abstract

Language agents increasingly operate over streams of related tasks, yet existing memory systems struggle to convert accumulated experience into reusable knowledge. Retrieval-augmented and structured memory methods record per-session observations effectively, but often couple acquisition and consolidation into a single online process, leaving the agent without a global view across sessions to discover recurring patterns, abstract shared procedures, or prune redundant entries. Inspired by complementary learning systems theory, we propose Auto-Dreamer, a learned offline consolidator for language-agent memory. Auto-Dreamer decouples fast per-session memory acquisition from slow cross-session consolidation. Given a selected working region of a typed memory bank, the consolidator treats the region as read-only evidence, performs bounded tool-use to inspect entries and provenance-linked source trajectories, and synthesizes a fresh compact replacement set that abstracts across sessions and supersedes the original region. We train Auto-Dreamer via GRPO, using end-to-end agent performance as the reward signal to learn how to consolidate memories acquired through fast online experience. Trained on ScienceWorld trajectories alone, Auto-Dreamer outperforms fixed, RL-trained, and prompted memory baselines on ScienceWorld by 7 points while using an active memory bank $12\times$ smaller than the strongest baseline, and continues to lead on held-out ALFWorld and WebArena without retraining — using $6\times$ less memory than the strongest baseline on ALFWorld.

1 Introduction

Language agents are increasingly deployed over streams of related tasks rather than isolated interactions [27, 33]. In such settings, long-term memory is not merely a retrieval cache for past entities or user preferences; it is the mechanism by which an agent converts raw experience into reusable procedures, environment knowledge, and behavioral priors that improve future decision making. A memory system must therefore solve two distinct problems: it must rapidly acquire useful information from each new trajectory, and it must periodically reorganize accumulated experience into a form that is compact, non-redundant, and useful for future tasks.

Recent work has made substantial progress on individual components of language-agent memory [10, 11], including retrieval-augmented episodic stores [43, 21], structured memory systems [29, 34], procedural skill libraries [6, 26, 20], reflection-based methods [23, 16], and RL-trained memory managers [37, 32, 35, 30]. Despite this progress, two challenges remain. First, a consolidation problem: existing methods typically couple acquisition and consolidation into a single online update process, so each update is made with limited evidence from the current session. This makes it difficult to discover recurring patterns, abstract reusable procedures that generalize across sessions, resolve contradictions, or prune redundant entries. Second, a memory-utility problem: RL-trained memory

^{*}Equal contribution. Order determined by coin flip; both authors reserve the right to list themselves as first author.

methods optimize online construction or retrieval rather than offline consolidation under an explicit downstream utility objective, so they do not directly learn which memories are load-bearing, which entries are redundant, or how to trade off success against memory compactness.

We take inspiration from complementary learning systems (CLS) theory of human memory, in which a fast hippocampal system encodes individual episodes and a slower neocortical system gradually extracts shared structure across episodes [17, 12, 18]. We adopt CLS not as a biological claim about language models, but as an operational design principle for separating fast acquisition from slow cross-session consolidation. We introduce **Auto-Dreamer**, a learned offline consolidator for language-agent memory.² Auto-Dreamer is the slow-timescale counterpart to a fast per-session writer. Given a typed memory bank produced by the writer, it performs a multi-step tool-use rollout: searching memory, inspecting candidate entries, retrieving raw source trajectories for provenance, and synthesizing new entries that abstract across sessions. Its core operation is *region rewriting*: the consolidator treats a selected working region as read-only evidence and synthesizes a fresh replacement set that supersedes the original region. This replacement semantics makes compactness structural rather than auxiliary: old entries do not persist by default, and information survives only if it is re-synthesized into the replacement set. As a result, abstraction, deduplication, contradiction resolution, and omission-based forgetting become default behaviors. We train Auto-Dreamer with GRPO [22] using a composite reward that combines downstream task performance with a counterfactual utility term estimated by random memory masking, which penalizes redundant entries while rewarding load-bearing ones. The task agent and the per-session writer remain fixed throughout training, isolating the contribution of the consolidator.

We evaluate Auto-Dreamer in two regimes: continual-memory deployment, where the bank starts empty and grows over the task stream, and fixed-bank consolidation, where a pre-built bank is rewritten once. The results support three conclusions. First, Auto-Dreamer improves task success while maintaining substantially smaller active memory banks: in continual deployment, it achieves 41.1% success on ScienceWorld [28], 7 points above the strongest baseline with $12\times$ less memory; 60.2% on held-out ALFWorld [24] with $6\times$ less memory than the strongest baseline; and 52.3% on held-out WebArena [44], leading all baselines. Second, the learned consolidator transfers beyond its training distribution: although trained only on ScienceWorld trajectories, it improves performance on held-out ALFWorld and WebArena without further updates, including across a writer-backbone shift from Qwen3-14B [25] to Gemini-3.1-flash-lite-preview [4]. Third, controlled fixed-bank experiments and ablations show that the gains come from offline consolidation itself: region rewriting improves the quality of a given memory bank, while the counterfactual utility term suppresses redundant memories without sacrificing task performance.

Our contributions are summarized as follows:

- **A two-timescale formulation of language-agent memory.** We distinguish fast per-session acquisition from slow cross-session consolidation and formulate the latter as a learned decision problem over accumulated evidence.
- **Region rewriting as a compactness-inducing consolidation primitive.** We formulate offline consolidation as provenance-grounded region rewriting: a selected working region is treated as read-only evidence and replaced by a synthesized replacement set. This differs from per-entry CRUD by making cross-session abstraction, deduplication, and omission-based forgetting the default update semantics.
- **RL training with region-local credit.** Because region rewriting produces a self-contained replacement set, we can evaluate it directly and assign local credit from downstream task performance without supervised memory labels. We further use counterfactual masking to favor load-bearing memories and suppress redundant entries, improving the task utility of the compact bank.

2 Related works

Memory systems for language agents. A growing body of work designs memory architectures for language agents. Early systems organize memory around atomic units or flat stores, such as A-MEM [34], Mem0 [2], MemOS [13], and SimpleMem [15]. More recent work introduces richer typed memory spanning episodic, semantic, and procedural stores, including EverMemOS [9], MIRIX [29], Nemori [19], and PlugMem [36]. A complementary line focuses on extracting reusable procedures or

²Auto-Dreamer is distinct from the Dreamer family of world models [7, 8]; our method operates on memory entries and source trajectories, not the latent environment dynamics.

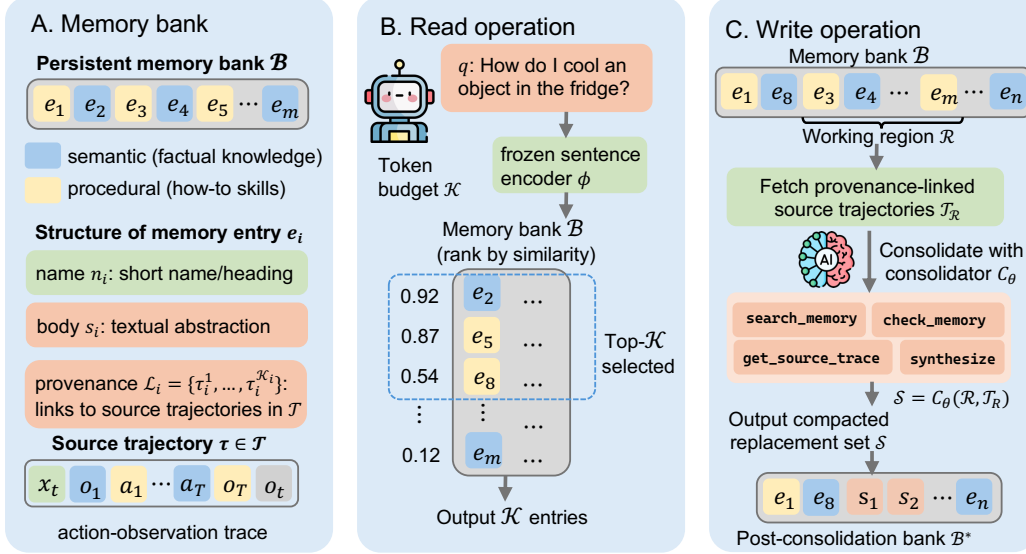


Figure 1: **Memory primitives and operations.** (A) The memory bank $\mathcal{B}$ holds typed entries (semantic or procedural); each entry has a short name n_i , a body s_i , and provenance links to source trajectories in the trajectory log $\mathcal{T}$. (B) The read operator retrieves the top- K entries by cosine similarity between a frozen sentence encoder ϕ applied to the query and to each entry’s name-body text. (C) The write operator applies a learnable consolidator C_θ to a working region $\mathcal{R} \subseteq \mathcal{B}$ and its provenance-linked trajectories $\mathcal{T}_{\mathcal{R}}$, producing a replacement set $\mathcal{S}$ that supersedes $\mathcal{R}$ in the post-consolidation bank $\mathcal{B}^*$.

strategies from trajectories: Memp [6] and Voyager [26] build procedural skill libraries, ExpeL [42] extracts cross-task insights from successful and failed trajectories, ReasoningBank [20] distills high-level reasoning strategies, and ReMem [32] studies test-time memory evolution. These systems improve how agents store experience, but their memory updates are governed by prompted heuristics applied within or immediately after each session, without explicit cross-session consolidation.

RL-trained memory managers. Recent work explores training language models to construct memory using reinforcement learning. MEM1 [45] and MemAgent [38] train models to update simple, text-only memories. Memory-R1 [35], Learn-to-Memorize [41], REMEMBER [39], and Mem- α [30] introduce richer memory representations and teach agents to manage complex memory systems through interaction and feedback. However, these methods primarily focus on teaching the model to extract and organize knowledge from its input, rather than on improving downstream agentic task performance. Later methods bridge this gap: UMEM [37] jointly trains memory extraction and management with GRPO under an online single-step interface, and MemRL [40] trains the agent to retrieve the correct memory at decision time. Nevertheless, all of these operate online: memory updates are interleaved with task execution, so consolidation evidence is limited to the current session. Auto-Dreamer addresses a complementary problem, operating offline over a bank accumulated across many sessions with access to the full memory bank and raw source trajectories.

Offline computation and sleep-time memory. Sleep-time compute [14] pre-computes over persistent context before queries arrive, amortizing reasoning across future interactions. LightMem [5] combines an online writer with periodic offline consolidation, but implements consolidation as a fixed prompted pipeline with per-entry CRUD decisions. Auto-Dreamer instead performs *region rewriting*: it treats a selected working region as read-only evidence, then uses a learned multi-step tool-using consolidator to synthesize a fresh compact replacement set that abstracts across sessions and supersedes the original region. The replacement set is grounded in re-readable source trajectories and trained with downstream task reward.

3 Preliminaries

Task setup. A frozen task agent operates over a stream of sessions τ , each yielding an action–observation trace and final outcome. The agent has access to a typed long-term memory bank $\mathcal{B}$ through a fixed retriever, and a trajectory log $\mathcal{T}$ that records raw sessions for provenance. Offline

consolidation leaves the task agent, retriever, and memory schema fixed; it transforms $\mathcal{B}$ into a post-consolidation bank $\mathcal{B}^*$. Raw trajectories in $\mathcal{T}$ are not retrieved by the task agent at decision time but remain available to the consolidator as provenance evidence.

Memory bank. A memory entry is a typed textual abstraction, with a short name n , a body s , and provenance links to source trajectories in $\mathcal{T}$. Each entry is either *semantic* (factual environment knowledge, e.g., “the toiletpaperhanger is typically on the bathroom wall”) or *procedural* (reusable how-to skills, e.g., “to cool an object, place it in the fridge, wait, then retrieve it”). The memory bank $\mathcal{B}$ is a set of such entries.

Memory operations. The bank supports two complementary operations: an online read operator used by the frozen task agent, and an offline write operator learned by the consolidator:

$$\text{Read}(q; \mathcal{B}) = \text{Top-}K_{e \in \mathcal{B}} \cos(\phi(q), \phi(n_e \oplus s_e)), \text{Write}(\mathcal{B}, \mathcal{R}, \mathcal{T}_{\mathcal{R}}) = (\mathcal{B} \setminus \mathcal{R}) \cup C_{\theta}(\mathcal{R}, \mathcal{T}_{\mathcal{R}}) \quad (1)$$

Here ϕ is a frozen sentence encoder, K is the largest ranked prefix fitting the token budget, $\oplus$ denotes string concatenation, $\mathcal{R} \subseteq \mathcal{B}$ is the rewritten memory region, and $\mathcal{T}_{\mathcal{R}}$ collects the source trajectories linked from entries in $\mathcal{R}$, accessible to the consolidator via provenance with $C_{\theta}(\mathcal{R}, \mathcal{T}_{\mathcal{R}}) = \mathcal{S}$ producing a replacement set, yielding $\mathcal{B}^* = (\mathcal{B} \setminus \mathcal{R}) \cup \mathcal{S}$. We refer to this write operation as *region rewriting*: entries in $\mathcal{R}$ are treated as read-only evidence rather than edited in place, and entries outside $\mathcal{R}$ are left unchanged.

Evaluation tasks. At training time, the consolidator is scored on an evaluation task set $\mathcal{V}$ randomly drawn from the training environment, disjoint from the trajectories used to construct training memory regions and from all tasks reported in Section 5.

4 Methodology

AUTO-DREAMER instantiates the offline consolidation problem (Section 3) with a learned tool-using policy. A fixed online writer provides fast acquisition by extracting typed memories from individual sessions. A learned offline consolidator provides slow consolidation by rewriting selected memory regions after experience has accumulated. The task agent, retriever, writer, memory schema, and token budget are all fixed; only the consolidator parameters are updated during training, and only the memory bank is updated after deployment. At a consolidation event, a region selector provides $\mathcal{R} \subseteq \mathcal{B}$. The consolidator C_{θ} performs a bounded tool-use rollout: searching the bank, inspecting candidate entries, and retrieving provenance-linked source trajectories, and emitting synthesized entries. The synthesized entries form a replacement set $\mathcal{S}$, inserted into the bank.

4.1 Designing Two-timescale Memory for Auto-Dreamer

Fast online acquisition. After each session, a prompted writer language model emits typed memory entries that record potentially useful experience, each storing a provenance pointer to the source trajectory. The writer is intentionally local and append-only: it does not search the existing bank, compare entries across sessions, or rewrite old memories. This favors plasticity—new experience is recorded immediately and cheaply. Slower cross-session operations (merging, abstraction, correction, compression, forgetting) are delegated to the consolidator. The writer prompt and schema are given in Appendix E.

Slow consolidation as region rewriting. A bank produced by repeated local writing is useful but inefficient. It may contain duplicate procedures, overspecific rules, stale facts, and partial observations whose common structure is only visible across sessions. AUTO-DREAMER addresses this by learning to rewrite active memory regions. Given a region $\mathcal{R} \subseteq \mathcal{B}$ and its provenance-linked trajectories $\mathcal{T}_{\mathcal{R}}$, the consolidator performs a bounded tool-use rollout over a fixed turn budget, with each step conditioned on $(\mathcal{R}, \mathcal{T}_{\mathcal{R}})$ and the history of previous tool calls and observations (tool interface in Appendix E.2). The rollout ends when the policy emits `terminate` or reaches the budget. The synthesized entries form the replacement set $\mathcal{S}$, and the bank is updated by replacing $\mathcal{R}$ with $\mathcal{S}$ according to Eq. 1.

This provenance-grounded region-replacement semantics is central to Auto-Dreamer’s compactness. In CRUD-style memory managers, existing memories persist unless the controller explicitly edits or deletes them, so consolidation is expressed as many local retention decisions. In region rewriting, the unit of rewriting is a region rather than an individual entry: the old entries serve as evidence, and only information re-synthesized into the replacement set remains active. This makes abstraction, deduplication, contradiction resolution, and omission-based forgetting the default behaviors of the

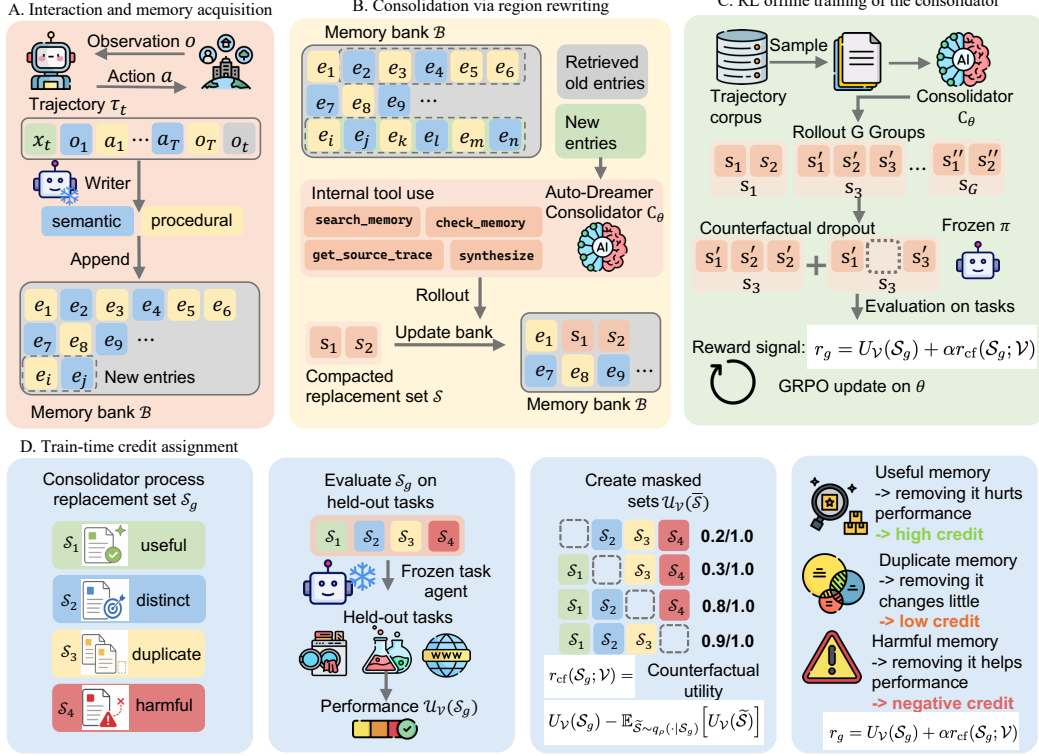


Figure 2: **Auto-Dreamer overview.** (A) A frozen writer appends typed entries from each trajectory τ_t to the memory bank $\mathcal{B}$. (B) Every k sessions, the consolidator C_θ rewrites a working region $\mathcal{R}$ into a replacement set $\mathcal{S}$ via tool-use rollout over memory and provenance trajectories. (C) Training: G group rollouts produce candidates $\{S_g\}$, scored on evaluation tasks $\mathcal{V}$; GRPO updates θ using reward $r_g = R_V(S_g) + \alpha r_{cf}(S_g; \mathcal{V})$. (D) The counterfactual term r_{cf} compares each S_g against masked variants, assigning high credit to useful entries, low credit to duplicates, and negative credit to harmful entries.

operator. As a result, compactness arises from the primitive itself, while learning determines which compact abstractions are most useful for downstream tasks.

4.2 Deploying Auto-Dreamer via Online Memory Acquisition

In the online setting, the trained consolidator updates the memory bank $\mathcal{B}$ using the Write operator from Eq. 1. We trigger consolidation every k sessions and define the working region $\mathcal{R}$ as the union of entries newly written during the interval and older entries retrieved by the task agent during the same interval. This working region bridges online memory acquisition and offline consolidation: newly written entries provide fresh evidence, while recently retrieved entries identify older memories currently interacting with the task agent’s behavior. The consolidator treats $\mathcal{R}$ as read-only evidence and synthesizes a replacement set $\mathcal{S}$, yielding $\mathcal{B}^* = (\mathcal{B} \setminus \mathcal{R}) \cup \mathcal{S}$. Entries outside $\mathcal{R}$ are left unchanged but may enter a future working region if retrieved in subsequent tasks.

4.3 Training Auto-Dreamer via Offline Memory Consolidation

We train the consolidator on regions sampled from an offline corpus of agent trajectories. We first collect trajectories from the training environments, run the fixed writer once, and store the resulting entries together with their provenance links. At each training step, we sample J support trajectories $\{\tau^{(j)}\}_{j=1}^J$ and form the working region $\mathcal{R}$ from their prewritten entries, with corresponding source trajectories $\mathcal{T}_{\mathcal{R}}$ accessible through provenance links. The consolidator samples G tool-use rollouts over $(\mathcal{R}, \mathcal{T}_{\mathcal{R}})$; rollout g produces a candidate replacement set S_g . For direct credit assignment, training evaluates each replacement set in a local bank consisting only of the synthesized entries: $\mathcal{B}_g^* = S_g$.

Algorithm 1 Auto-Dreamer Consolidator Training

Require: Offline pool of trajectories with prewritten entries; frozen task agent; evaluation set $\mathcal{V}$; support size J ; group size G ; rollout budget $T_{\max}$

- 1: **for** each training step **do**
- 2: Sample J support trajectories $\{\tau^{(j)}\}_{j=1}^J$ from the offline pool
- 3: Form working region $\mathcal{R} \leftarrow$ entries written from $\{\tau^{(j)}\}$, with provenance trajectories $\mathcal{T}_{\mathcal{R}}$
- 4: **for** $g = 1, \dots, G$ **do**
- 5: $\mathcal{S}_g \leftarrow C_{\theta}(\mathcal{R}, \mathcal{T}_{\mathcal{R}})$ // tool-use rollout, up to $T_{\max}$ steps
- 6: $r_g \leftarrow U_{\mathcal{V}}(\mathcal{S}_g) + \alpha r_{\text{cf}}(\mathcal{S}_g; \mathcal{V})$ // Eq. 2, local bank $\mathcal{B}_g^* = \mathcal{S}_g$
- 7: **end for**
- 8: Update θ via GRPO using $\{r_g\}_{g=1}^G$
- 9: **end for**

Local evaluation for credit assignment. Region rewriting turns consolidation into a locally evaluable operator-learning problem. Each rollout produces a self-contained replacement set, so we evaluate it in a local bank consisting only of the synthesized memories $\mathcal{B}_g^* = \mathcal{S}_g$. This aligns the unit of credit assignment with the unit produced by the policy: reward is assigned directly to the replacement set, rather than to a full bank whose performance may be explained by memories not produced by the current rollout. In this way, we learn a region-local improvement operator. At deployment, the same learned operator rewrites a selected working region, while entries outside the working region are left unchanged, as in Eq. 1. The consolidation interface, provenance tools, schema, frozen writer, retriever, task agent, and memory-token budget are shared across training and deployment. Our continual-memory deployment experiments in Section 5.2 show that this region-local objective transfers to persistent-bank composition.

We train the consolidator C_{θ} with GRPO [22]. The writer, retriever, task agent, and memory-token budget are fixed across rollouts; only the consolidator parameters θ are updated. The consolidator is initialized from Qwen3-14B. Per-environment data construction, rollout budgets, and optimization hyperparameters are given in Appendix B.3.

Reward design. For rollout g , the reward combines downstream performance with a counterfactual estimate of memory utility:

$$r_g = U_{\mathcal{V}}(\mathcal{S}_g) + \alpha r_{\text{cf}}(\mathcal{S}_g; \mathcal{V}). \quad (2)$$

The two terms are defined as

$$U_{\mathcal{V}}(\mathcal{S}) = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \text{Return}(v; \mathcal{S}) \quad r_{\text{cf}}(\mathcal{S}_g; \mathcal{V}) = U_{\mathcal{V}}(\mathcal{S}_g) - \mathbb{E}_{\tilde{\mathcal{S}} \sim q_{\rho}(\cdot | \mathcal{S}_g)} [U_{\mathcal{V}}(\tilde{\mathcal{S}})]. \quad (3)$$

Here α weights the counterfactual term, and the distribution $q_{\rho}(\tilde{\mathcal{S}} | \mathcal{S}_g)$ masks a fixed fraction ρ of entries from $\mathcal{S}_g$ uniformly at random. In practice, the expectation in r_{cf} is estimated with M_g Monte Carlo samples. The counterfactual term measures the expected performance drop under random ablation of the synthesized replacement set: masking load-bearing entries lowers performance, masking redundant entries has little effect because duplicate information remains, and masking harmful entries can improve performance, making r_{cf} negative. In the GRPO update, this favors replacement sets whose entries improve downstream utility with minimal redundancy.

5 Experiments

We evaluate Auto-Dreamer along three axes. Section 5.2 studies realistic continual-memory deployment, showing improved task success over per-session memory baselines with a compact memory bank. Section 5.3 isolates the consolidation operator in a fixed-bank setting and shows gains over prompted offline baselines. Section 5.4 ablates the key design choices, highlighting the roles of offline consolidation and the counterfactual utility reward.

5.1 Experimental Settings

Tasks. We evaluate Auto-Dreamer on three tasks in different domains and difficulty: *ALFWorld* [24] (household instruction-following), *ScienceWorld* [28] (text-based science experiments), and *WebArena* [44] (web navigation; shopping, shopping_admin, gitlab).

Table 1: **Memory evaluation across continual deployment and controlled consolidation.** Panel A reports continual-memory deployment where memory starts empty and is updated from trajectories collected during evaluation. Panel B isolates the consolidation operator by giving each method the same fixed initial bank $\mathcal{B}_0$ and evaluating on held-out tasks with a frozen task agent. For SR and AUC, higher is better; for #Tok., lower is better. **Bold** and underline denote the best and second-best results.

(A) Continual-memory deployment										(B) Bank consolidation: Control study					
Method	ALFWorld			ScienceWorld			WebArena			Method	ALFWorld		ScienceWorld		
	SR	#Tok.	AUC	SR	#Tok.	AUC	SR	#Tok.	AUC		SR	#Tok.	SR	#Tok.	
<i>Baseline</i>										<i>Baseline</i>					
No memory	30.8	0	0.287	28.7	0	0.295	44.6	0	0.527	No memory	24.5	0	29.6	0	
<i>Reflective / insight extraction</i>										<i>Reflective / insight extraction</i>					
Reflexion [23]	49.2	11,967	0.475	29.6	49,936	0.306	46.4	8,148	0.567	Reflexion [23]	46.3	455	31.0	661	
ExpeL [42]	33.6	2,042	0.306	28.3	11,628	0.289	50.8	3,371	0.576	ExpeL [42]	<u>70.1</u>	2,942	38.1	808	
<i>Workflow / procedural memory</i>										<i>Workflow / procedural memory</i>					
AWM [31]	32.8	16,846	0.306	30.2	3,877	0.311	<u>52.0</u>	890	<u>0.597</u>	AWM [31]	66.9	758	33.6	103	
Memp [6]	31.4	<u>6,731</u>	0.297	30.4	11,531	0.306	50.8	4,973	0.574	Memp [6]	67.3	927	40.7	351	
<i>Structured stores</i>										<i>Structured stores</i>					
ReasoningBank [20]	31.1	42,784	0.285	30.9	155,117	0.324	49.8	24,362	0.581	ReasoningBank [20]	43.8	3,021	25.0	3,221	
Mem0 [2]	30.6	119,013	0.279	26.8	89,854	0.281	50.3	43,355	0.571	Mem0 [2]	66.1	4,953	<u>42.0</u>	1609	
<i>Two-timescale prompted</i>										<i>Two-timescale prompted</i>					
LightMem [5]	31.2	130,001	0.288	28.1	272,074	0.287	<u>52.0</u>	370,874	0.567	LightMem [5]	40.6	243	31.8	<u>215</u>	
<i>RL-trained writers</i>										<i>RL-trained writers</i>					
Mem- α [30]	57.4	125,635	<u>0.566</u>	30.0	344,599	0.297	— [†]	— [†]	— [†]	Mem- α [30]	56.6	1,676	25.7	1,142	
UMEM [37]	<u>58.4</u>	62,947	0.564	<u>34.1</u>	80,918	<u>0.353</u>	— [†]	— [†]	— [†]	UMEM [37]	54.2	5,184	29.3	1,298	
<i>Ours</i>										<i>Ours</i>					
Auto-Dreamer	60.2	10,954	0.585	41.1	<u>6,947</u>	0.411	52.3	<u>927</u>	0.628	Auto-Dreamer	72.7	634	44.3	539	

[†]UMEM and Mem- α rely on small open-source memory-optimizer models whose context windows cannot accommodate WebArena’s accessibility-tree observations together with the memory bank. We do not retrain with larger backbones because doing so departs from the published configurations and exceeds our compute budget.

Models. The frozen task agent is shared across all methods within a domain: Qwen3.5-9B on ALFWorld and ScienceWorld, and Gemini-3-flash-preview [3] on WebArena. For RL-trained memory baselines, we use the released 4B Mem- α checkpoint and a Qwen3-14B UMEM model trained on ALFWorld trajectories. All other baselines and Auto-Dreamer’s per-session writer use Qwen3-14B on ALFWorld and ScienceWorld, and Gemini-3.1-flash-lite-preview [4] on WebArena, ensuring no baseline is disadvantaged by a weaker memory-generation LLM. The Auto-Dreamer consolidator C_θ is initialized from Qwen3-14B, trained on ScienceWorld trajectories only, and applied without further updates on all three domains—including across the writer-backbone shift on WebArena.

Baselines. Ten baselines spanning seven families: no memory (No memory); reflective and insight extraction (Reflexion [23], ExpeL [42]); workflow and procedural memory (AWM [31], Memp [6]); structured stores (ReasoningBank [20], Mem0 [2]); two-timescale prompted memory (LightMem [5], the closest architectural counterpart to our method); RL-trained writers (Mem- α [30], UMEM [37]). Detailed descriptions and per-baseline implementation are in Appendix B.1.

Metrics. We report task success rate (SR, %), macro-averaged over task families within each domain, and final active memory-bank size in tokens (#Tok). For continual-memory deployment, we additionally report the normalized area under the cumulative success curve ($AUC \in [0, 1]$) over the task stream.

5.2 Main Results

We evaluate Auto-Dreamer in the continual-memory deployment regime, where memory starts empty and is updated from trajectories collected during evaluation. After each completed session, the writer adds entries; methods with offline consolidation invoke their updater at a fixed cadence of k sessions, with values reported in Appendix B.2.

Auto-Dreamer improves success while keeping memory compact. Table 1 shows that Auto-Dreamer achieves the strongest overall continual-memory performance across all three domains. On ScienceWorld, it reaches 41.1% SR, improving over the strongest baseline UMEM by 7.0 points (34.1%) and over the strongest prompted baseline ReasoningBank by 10.2 points (30.9%). Its AUC also improves from 0.353 to 0.411, indicating that the gain appears throughout the stream rather than only at the end. The same pattern holds on ALFWorld, where Auto-Dreamer achieves 60.2% SR

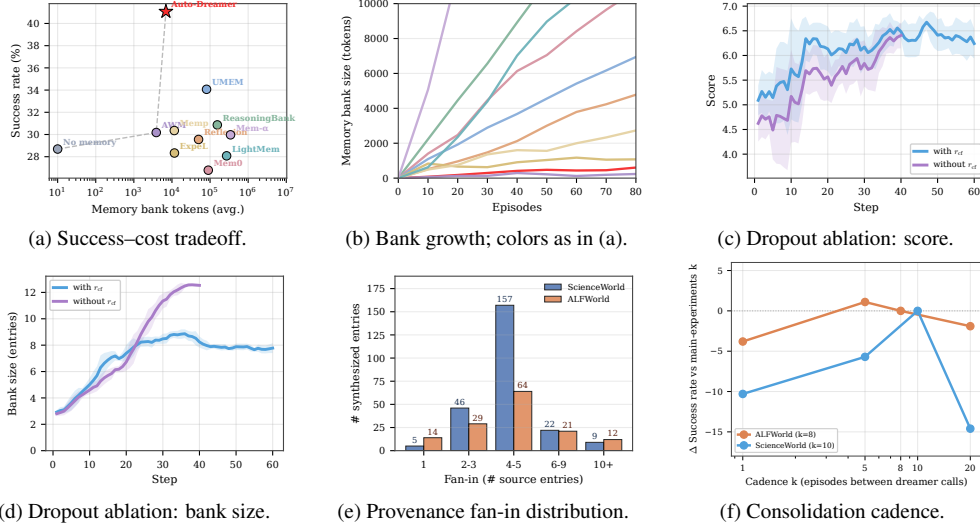


Figure 3: **Memory efficiency, reward ablation, and consolidator analysis.** (a) Auto-Dreamer lies on the Pareto frontier of task success versus retrieval-time memory cost. (b) Auto-Dreamer maintains a compact memory bank while most baseline methods grow monotonically as the task stream lengthens. (c,d) The counterfactual utility reward preserves task performance while bounding bank growth during training. (e) Provenance fan-in distribution peaks at fan-in = 5 on both ScienceWorld and ALFWorld, indicating that synthesized entries typically draw on multiple source memories rather than being one-to-one copies. (f) Consolidation cadence sweep: ScienceWorld performs best at the main cadence, while ALFWorld is comparatively robust across settings.

compared with 58.4% for UMEM and 57.4% for Mem- α , and on WebArena, where it obtains the highest SR despite the longer-horizon tasks and noisier accessibility-tree observations.

These gains are achieved with substantially lower retrieval-time memory cost. On ScienceWorld, Auto-Dreamer uses 6.9k memory tokens, compared with 80.9k for UMEM and 155.1k for ReasoningBank. On WebArena, it uses only 927 tokens, compared with 370k for LightMem and 43.4k for Mem0, while still obtaining the best SR and AUC. Figure 3a summarizes this tradeoff: baselines that approach Auto-Dreamer’s success typically require much larger banks, whereas compact baselines generally sacrifice success. Auto-Dreamer therefore occupies the favorable region of the success–cost plane.

The learned consolidator transfers across task domains and writer backbones. Auto-Dreamer’s consolidator is trained only on ScienceWorld trajectories, yet it improves continual-memory performance on held-out ALFWorld and WebArena without additional updates. This demonstrates transfer not only across task domains, but also across memory-acquisition backbones: the same trained consolidator is paired with a Qwen3-14B writer on ALFWorld and ScienceWorld and with a Gemini-3.1-flash-lite-preview writer on WebArena. This supports the domain- and writer-agnostic design of the consolidation interface in §4.1: the consolidator operates over typed textual memory entries and provenance-linked trajectory excerpts, rather than environment-specific state representations, action symbols, or writer-specific hidden states.

The online results also test the training–deployment approximation in §4.3. Although training evaluates rewritten regions locally for credit assignment, Panel A evaluates repeated composition into a persistent, growing bank. Auto-Dreamer’s gains show that the locally trained rewrite operator remains effective under retrieval competition and interaction with older memories.

5.3 Discussion on Bank Consolidation

The continual-memory deployment in §5.2 measures end-to-end performance with many entangled factors: writer quality, retrieval competition over a growing bank, consolidation cadence, and downstream agent decisions over many sessions. We complement it with a controlled study that isolates the consolidation operator itself.

Setup. Each method receives an identical fixed initial bank $\mathcal{B}_0$ constructed in advance from a sampled pool of trajectories run through the same fixed writer. The consolidator is invoked exactly once on $\mathcal{B}_0$, producing a consolidated bank $\mathcal{B}^*$. We then evaluate the frozen task agent equipped with $\mathcal{B}^*$ on a held-out task set drawn from the same environment family. Methods that lack a consolidation step (e.g., Reflexion, AWM) are evaluated directly on $\mathcal{B}_0$. This setup mirrors the training distribution of C_θ (§4.3) and decouples consolidation quality from the dynamics of an evolving stream. We use ALFWorld and ScienceWorld; WebArena is omitted because the long-horizon, stateful nature of web tasks precludes constructing a meaningful fixed-bank evaluation that does not collapse into a stream.

Auto-Dreamer also leads in the controlled regime. Panel B of Table 1 reproduces the online ranking: Auto-Dreamer reaches 72.7% on ALFWorld and 44.3% on ScienceWorld, leading the strongest baseline on each (ExpeL at 70.1%, Mem0 at 42.0%) by 2.6 and 2.3 points respectively, with comparable or smaller memory cost. The narrower margin compared to continual deployment is consistent with the controlled regime removing the cumulative retrieval-competition effects that further amplify Auto-Dreamer’s advantage at deployment time.

5.4 Ablation Studies

Ablating offline consolidation. We ablate offline consolidation with two variants: *writer-only*, which keeps the same task agent, retriever, schema, and per-session writer but disables consolidation; and *untrained*, which uses the same region rewriting mechanism, tool-use rollout, and provenance grounding but without GRPO training.

Table 2 shows that the untrained pipeline already accounts for most of the bank-size reduction over writer-only ($6\text{--}11\times$ smaller across all three domains), supporting the view that region rewriting is the primary compactness mechanism: replacing a region with a synthesized set induces compression before learning. GRPO training then changes the quality and selectivity of the rewrite policy. On ScienceWorld and WebArena, training adds substantial SR gains (+9.7pp and +5.7pp); on ALFWorld, the aggregate gain is smaller (+1.0pp) and the active bank grows. Task family-level analysis reveals heterogeneous effects within ALFWorld: training helps on multi-step manipulation tasks but hurts on the single location-sensitive family `look_at_obj_in_light`, where task-specific location details are lost under abstraction. Excluding this family widens the trained-vs-untrained SR margin on the remaining five categories to +4.9pp, and we analyze this failure mode as Pattern 3 in §5.5.

Table 2: **Effect of offline consolidation.** Untrained Auto-Dreamer uses the same consolidator pipeline (region rewriting, tool-use rollout, provenance grounding) but without RL updates, comparing trained vs. untrained isolates the contribution of GRPO training.

Method	ALFWorld		ScienceWorld		WebArena	
	SR	#Tok.	SR	#Tok.	SR	#Tok.
Writer-only	48.0	50,735	29.6	61,679	44.2	6,421
Auto-Dreamer (untrained)	59.2	6,900	31.4	5,700	46.6	1,067
Auto-Dreamer	60.2	10,954	41.1	6,947	52.3	927

Ablating counterfactual utility reward. We next train Auto-Dreamer with and without the counterfactual utility term r_{cf} . Figures 3c and 3d report the resulting training dynamics. Without r_{cf} , the consolidator continues to improve raw environment score, but the bank grows rapidly over training. With r_{cf} , bank size first grows and then shrinks as training proceeds, while task performance remains competitive with the unshaped variant. This supports the intended role of counterfactual utility: it encourages compact, load-bearing memory banks without a measurable cost in task success.

Ablating consolidation cadence. We sweep the consolidation cadence $k \in \{1, 5, 20\}$ on ALFWorld and ScienceWorld, holding all other factors fixed. Figure 3f reports ΔSR relative to the main-experiment cadence ($k = 8$ on ALFWorld, $k = 10$ on ScienceWorld). ScienceWorld shows a larger cadence effect, with the main cadence outperforming both more frequent and less frequent consolidation. Performance is lower when consolidation is too frequent ($k = 1$), suggesting that very small intervals provide insufficient cross-session evidence for useful abstraction. Performance also drops when consolidation is too infrequent ($k = 20$), consistent with the consolidator being asked to process a larger and noisier working region that strains its context and tool-use budget. ALFWorld is

more robust to cadence, with a range of roughly 5pp; $k = 5$ slightly outperforms the main setting by 1.1pp.

5.5 Qualitative Analysis

Multi-source consolidation. Figure 3e reports the distribution of *provenance fan-in*, the number of source entries cited by each synthesized output. On both ScienceWorld and ALFWorld, the distribution peaks at fan-in = 5, indicating that synthesized entries typically draw on multiple source memories rather than being one-to-one copies. We next examine three qualitative patterns: two successful patterns drawn from matched tasks where the same agent succeeds with the Auto-Dreamer bank but fails with the writer-only bank, and one failure pattern where abstraction discards task-specific details.

Case Study: Slot Abstraction (find-entity, task 90)

Task: Find a living thing and place it in the **red box** in the **kitchen**.

Writer memory (3 hardcoded entries)

- ✗ “move to **blue box** in **living room**”
- ✗ “move to **green box** in **bathroom**”
- ✗ “move to **green box** in **bathroom**” (duplicate)

Task agent (fail, 8 steps):

- ✗ No memory matches (red, kitchen); agent searches workshop freezer (no living thing inside)
- ✗ Wanders to kitchen, focuses on an **orange** (a fruit, not living)
- ✗ Score -1.0

Auto-Dreamer memory

- ✓ “move to designated container (yellow, red, purple, or orange box)”
- ✓ “teleport to target room (bedroom, workshop, etc.)”
- ✓ “living things typically outside, greenhouse, or art studio”

Task agent (success, 6 steps):

- ✓ Goes to greenhouse, finds cherry tree (a living thing)
- ✓ Carries cherry tree to kitchen, places in red box
- ✓ Score 1.0

Pattern 2: filtering wrong and contradicting entries via abstraction. Writer entries from different past tasks can encode mutually contradictory claims about the same task element, or contain phrasing errors carried over from an LLM-generated trace. The agent, treating writer memory as authoritative, can act on a wrong entry and become stuck. Rather than adjudicating among conflicting specifics or propagating phrasing errors, the consolidator drops these entries and emits a higher-level rule that preserves the shared task structure while leaving instance-specific answers to in-context reasoning.

Case Study: Filtering Wrong Entries (lifespan-compare, task 61)

Task: Find the animal with the **longest**, then the **shortest**, lifespan.

Writer memory (contradictory + wrong)

- ✗ “longest is **crocodile**”
- ✗ “longest is **sea turtle**” (contradicts above)
- ✗ “shortest is mouse”
- ✗ “focus on adult **and** baby elephant” (writer error: trace only focused adult)

Task agent (fail, 18 steps):

- ✗ Follows the elephant entry; outputs action focus on adult adult elephant
- ✗ Env: “no such entity”; agent retries 16 times, never adapts
- ✗ Step cap reached, score -1.0

Auto-Dreamer memory

- ✓ “compare lifespans of the listed candidates”
- ✓ “focus on longest first, then shortest”
- ✓ “focus the adult life stage”

Task agent (success, 3 steps):

- ✓ Identifies elephant as the longest-lived candidate, focuses on it
- ✓ Identifies dragonfly as the shortest-lived, focuses on it
- ✓ Score 1.0

Pattern 3 (failure mode): over-compression of locally useful details. We compare Auto-Dreamer with the untrained variant. Both perform region-rewriting, and they differ in whether the consolidator policy is trained. Although the trained consolidator improves performance in most task families, it underperforms the untrained consolidator on *look_at_obj_in_light*. In this task, concrete locations of target object and light source can help disambiguate where to search and where to examine the object. These details are episodic in form, but can still be useful for the current task. The trained consolidator tends to replace such task-specific paths with generic procedural entries, while

the untrained consolidator retains more task-specific details. This suggests that the learned policy can over-compress specific facts that are locally useful. Quantitatively, this single category drives the bulk of the ALFWorld trained-vs-untrained gap in Table 2: removing `look_at_obj_in_light` widens the SR margin on the remaining five families from +1.0pp to +4.9pp (65.8 vs. 60.9).

Case Study: Over-Abstraction (`look_at_obj_in_light`, task 308)

Task: Examine the alarmclock under the desk lamp (alarmclock on desk-2, desk lamp on desk-1).

Auto-Dreamer (untrained) memory

- ✓ “alarmclock on desk-2, desk lamp on desk-1; take then go”
- ✓ “avoid repeated examine of desk-1 when target is on desk-2”

Task agent (success):

- ✓ Takes alarmclock to desk-1, examines under desk lamp
- ✓ score 1.0

Auto-Dreamer memory

- ✗ “Key considerations for interacting with light sources”
- ✗ “Step-by-step process for examining an object”

Task agent (fail):

- ✗ Lacks location info; loops examining wrong desk
- ✗ score −1.0

6 Conclusion

We presented **Auto-Dreamer**, a two-timescale memory system that pairs fast per-session writing with learned offline region rewriting. By separating online acquisition from slow cross-session consolidation, Auto-Dreamer matches or exceeds the strongest memory baselines on ALFWorld, ScienceWorld, and WebArena while maintaining substantially smaller active memory banks. A consolidator trained only on ScienceWorld transfers to held-out domains and across a writer-backbone shift, supporting the view that consolidation over textual memory entries and source trajectories can be domain- and writer-agnostic. Several extensions follow naturally. The current consolidator rewrites one working region per event; future work could maintain longer-range bank structure, jointly optimize retrieval and consolidation, or handle multimodal source trajectories. More broadly, offline learned consolidation may be useful whenever agent experience accumulates faster than it can be reorganized in-session.

References

- [1] Xiusi Chen, Shanyong Wang, Cheng Qian, Hongru Wang, Peixuan Han, and Heng Ji. DecisionFlow: Advancing large language model as principled decision maker. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 16668–16692. Association for Computational Linguistics, November 2025. ISBN 979-8-89176-335-7.
- [2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. In *European Conference on Artificial Intelligence (ECAI)*, 2025.
- [3] Google DeepMind. Gemini 3 Flash model card. Technical report, Google DeepMind, December 2025. URL <https://deepmind.google/models/model-cards/gemini-3-flash/>.
- [4] Google DeepMind. Gemini 3.1 Flash-Lite model card. Technical report, Google DeepMind, March 2026. URL <https://deepmind.google/models/model-cards/gemini-3-1-flash-lite/>.
- [5] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. LightMem: Lightweight and efficient memory-augmented generation. *arXiv preprint arXiv:2510.18866*, 2025.
- [6] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*, 2025.

- [7] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.
- [8] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [9] Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, et al. Evermemos: A self-organizing memory operating system for structured long-horizon reasoning. *arXiv preprint arXiv:2601.02163*, 2026.
- [10] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, et al. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*, 2025.
- [11] Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, et al. Rethinking memory mechanisms of foundation agents in the second half. *arXiv preprint arXiv:2602.06052*, 2026.
- [12] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016. doi: 10.1016/j.tics.2016.05.004.
- [13] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, et al. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724*, 2025.
- [14] Kevin Lin, Charlie Snell, Yu Wang, Charles Packer, Sarah Wooders, Ion Stoica, and Joseph E. Gonzalez. Sleep-time compute: Beyond inference scaling at test-time. *arXiv preprint arXiv:2504.13171*, 2025.
- [15] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Simplemem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*, 2026.
- [16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- [17] James L. McClelland, Bruce L. McNaughton, and Randall C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3): 419–457, 1995. doi: 10.1037/0033-295X.102.3.419.
- [18] James L McClelland, Bruce L McNaughton, and Andrew K Lampinen. Integration of new information in memory: new insights from a complementary learning systems perspective. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 375(1799), 2020.
- [19] Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. Nemori: Self-organizing agent memory inspired by cognitive science. *arXiv preprint arXiv:2508.03341*, 2025.
- [20] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025.
- [21] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [22] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

- [23] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [24] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*, 2021.
- [25] Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [26] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [27] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers Comput. Sci.*, 18(6): 186345, 2024. doi: 10.1007/S11704-024-40231-1. URL <https://doi.org/10.1007/s11704-024-40231-1>.
- [28] Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. ScienceWorld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.
- [29] Yu Wang and Xi Chen. MIRIX: Multi-agent memory system for LLM-based agents. *arXiv preprint arXiv:2507.07957*, 2025.
- [30] Yu Wang, Ryuichi Takanobu, Zhiqi Liang, Yuzhen Mao, Yuanzhe Hu, Julian McAuley, and Xiaojuan Wu. Mem- α : Learning memory construction via reinforcement learning. *arXiv preprint arXiv:2509.25911*, 2025.
- [31] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [32] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. Evo-Memory: Benchmarking LLM agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025.
- [33] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, Qi Zhang, and Tao Gui. The rise and potential of large language model based agents: a survey. *Sci. China Inf. Sci.*, 68(2), 2025. doi: 10.1007/S11432-024-4222-0. URL <https://doi.org/10.1007/s11432-024-4222-0>.
- [34] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [35] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.
- [36] Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. Plugmem: A task-agnostic plugin memory module for llm agents. *arXiv preprint arXiv:2603.03296*, 2026.
- [37] Yongshi Ye, Hui Jiang, Feihu Jiang, Tian Lan, Yichao Du, Biao Fu, Xiaodong Shi, Qianghuai Jia, Longyue Wang, and Weihua Luo. UMEM: Unified memory extraction and management framework for generalizable memory. *arXiv preprint arXiv:2602.10652*, 2026.

- [38] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, et al. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. *arXiv preprint arXiv:2507.02259*, 2025.
- [39] Danyang Zhang, Lu Chen, Situo Zhang, Hongshen Xu, Zihan Zhao, and Kai Yu. Large language models are semi-parametric reinforcement learning agents. *Advances in Neural Information Processing Systems*, 36:78227–78239, 2023.
- [40] Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, et al. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026.
- [41] Zeyu Zhang, Quanyu Dai, Rui Li, Xiaohe Bo, Xu Chen, and Zhenhua Dong. Learn to memorize: Optimizing llm-based agents with adaptive memory framework. *arXiv preprint arXiv:2508.16629*, 2025.
- [42] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [43] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731, 2024.
- [44] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2307.13854>.
- [45] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*, 2025.

A Limitations

Evaluation scope. Our evaluation is restricted to three text-based agent environments sharing an LLM-mediated interface (ALFWorld, ScienceWorld, WebArena). We make no claims about transfer to settings with structured state representations, non-textual observations [1], or domains where memory must encode visual or multimodal evidence.

Writer and schema dependence. The consolidator operates over entries written by a fixed prompted writer with a specific semantic/procedural schema. We hold these constant to isolate the consolidator, but robustness to alternative writers, schemas, or noisier provenance links remains untested. Information missed by the writer cannot generally be recovered by the consolidator unless source trajectories make it salient.

Retrieval-budget sensitivity. Our main evaluation uses top- $K=3$ retrieval with a token cap on retrieved entries. This regime favors compact banks; methods that benefit from larger retrieval budgets may rank differently under looser constraints. Characterizing how the Pareto frontier shifts with the retrieval budget is left to future work.

Surrogate training objective. The local-bank training objective (§4.3) is a surrogate for deployment-time bank composition. While our online experiments show that the surrogate transfers, the formal relationship between local-bank ranking and full-bank ranking is not characterized.

Variance. We report point estimates without seed or task-order variance. Several margins in Table 1, particularly on ALFWorld, are small enough that variance estimates would be useful for interpretation.

B Experimental Details

B.1 Baseline Details

We compare AUTO-DREAMER against ten memory mechanisms for LLM agents, spanning six families. For each baseline we describe what is written into memory, how it is retrieved, and how it is consolidated. Unless otherwise noted, we use the original authors’ prompts and hyperparameters, and pair each method with the same backbone task agent for fairness.

No memory. A memoryless baseline in which the task agent receives only the current observation and the task instruction. It serves as a baseline that isolates the contribution of any memory mechanism on top of the underlying policy.

B.1.1 Reflective / Insight Extraction

Reflexion [23]. After each trajectory, the agent generates a free-form natural-language *reflection* that diagnoses failures and proposes corrections. Reflections are appended to a per-task buffer and prepended to the prompt on subsequent attempts. There is no cross-task generalization or structured retrieval: memory is task-local and grows monotonically until truncated by the context budget.

ExpeL [42]. ExpeL distills successful and failed trajectories into a small set of high-level *insights* (rules of thumb) and a pool of in-context exemplars. At inference time, the most relevant insights and exemplars are retrieved by similarity to the current task and inserted into the prompt. Compared to Reflexion, ExpeL transfers across tasks and emphasizes compact, generalizable rules over per-episode reflections.

B.1.2 Workflow / Procedural Memory

AWM (Agent Workflow Memory) [31]. AWM induces reusable *workflows* – abstracted action templates extracted from successful trajectories – and stores them in a workflow library. On a new task, the most relevant workflows are retrieved and injected as procedural guidance for the agent. Memory growth is tied to the diversity of induced workflows rather than to the number of trajectories, leading to compact stores.

Memp [6]. Memp builds a procedural memory by summarizing trajectories into stepwise *procedures* and indexing them for retrieval. It supports both addition and revision of procedures as new evidence

accumulates, occupying a middle ground between purely episodic stores (Reflexion) and abstract workflow libraries (AWM).

B.1.3 Structured Stores

ReasoningBank [20]. ReasoningBank maintains a structured bank of *reasoning traces* extracted from prior episodes, organized to support semantic retrieval. Each entry captures the chain-of-thought and key decision points of a trajectory, which are surfaced to the agent on related future tasks. The store grows quickly with experience, trading retrieval coverage for substantial token overhead.

Mem0 [2]. Mem0 is a general-purpose long-term memory layer that extracts atomic *facts* and *preferences* from interactions and stores them in a queryable memory graph. Retrieval combines vector similarity with light structural reasoning over the graph. We adapt Mem0 to the agentic setting by treating each trajectory as an interaction stream from which memories are distilled.

B.1.4 Two-Timescale Prompted

LightMem [5]. LightMem separates memory operations into two timescales: a fast *working memory* that buffers recent context, and a slower *consolidation* step that periodically distills the buffer into long-term notes. Both stages are fully prompted, with no learned components. This yields a clean ablation point for two-timescale designs that does not rely on reinforcement learning.

B.1.5 RL-Trained Writers

Mem- α [30]. Mem- α trains the *memory writer* with reinforcement learning, optimizing what to write so that downstream task success is maximized. The reader/retrieval pipeline is held fixed, isolating the contribution of a learned write policy. We omit Mem- α on WebArena: the released checkpoint relies on a small 4B memory-optimizer backbone whose context window cannot accommodate WebArena’s accessibility-tree observations together with the memory bank (see footnote in Table 1).

UMEM [37]. UMEM (Unified Memory) similarly trains the writer with RL but unifies episodic, procedural, and semantic memory into a single store with a learned update operator. It represents the strongest RL-trained baseline in our comparison. As with Mem- α , we omit UMEM on WebArena due to context window constraints.

B.1.6 Baseline Implementation

All baselines share the Qwen2.5-3B last-token-hidden embedder (2048-d) and FAISS IndexFlatIP retrieval over L2-normalized vectors, exposing memory through a common `read()/write()/reflect()` interface. Table 3 lists what we changed and what we preserved verbatim from each method’s released code; commit hashes are recorded in the top-of-file docstring of every `baselines/*/memory.py`. Hyperparameters are gathered in Table 4, and verbatim memory-construction prompts in Appendix G.

B.2 Evaluation Settings

Evaluation protocol. We adopt a *prequential (online streaming)* protocol implemented in `online_memory.eval.run_online`: held-out tasks are presented to the agent one at a time in a fixed seeded order; before task t , the agent retrieves from the bank built from tasks $1, \dots, t-1$, and after the task its trajectory is fed to the writer (and, on cadence, to the dreamer) so that future tasks see whatever new entries this trajectory produced. No task is ever replayed.

Consolidation cadence. Methods with offline consolidation invoke their updater at a fixed cadence of k completed sessions. We use $k=10$ for ScienceWorld, $k=8$ for ALFWorld, and $k=5$ for WebArena. The same cadence is used for all offline-consolidation methods within each environment.

Environments and task pools. We evaluate on three long-horizon agent environments through a single `EnvAdapter` interface so that the agent loop, retrieval pipeline, and memory store are byte-identical across domains (Table 5). Tasks are deterministically seeded with `--seed 42`; ALFWorld

Table 3: **Per-baseline modifications relative to each method’s released code.**

Baseline	Adapted	Preserved verbatim
ReasoningBank	Encoder swapped to Qwen2.5-3B; LLM-as-Judge success label replaced by the env’s gold reward; MaTTS scaling not implemented.	Successful / failed extraction prompts and the memory-injection banner (Tabs. 8–10), from commit ea65efd2.
ExpeL	Encoder swapped to Qwen2.5-3B; few-shot count raised from 2 to 10.	Compare-critique, all-success, and rule-operation templates (Tabs. 12–11).
LightMem	Encoder swapped to Qwen2.5-3B; topic-segmentation (paper §3.2) not replicated.	Three-stage sensory→STM→LTM pipeline with LLMingua-2 compression; metadata-extraction and consolidation prompts (Tabs. 14–15).
Mem0	Encoder swapped to Qwen2.5-3B; mem0.Memory() bypassed (upstream drifted to a V3 additive pipeline); driven directly with the legacy ADD/UPDATE/DELETE/NONE prompts from commit 3fbc1c9a.	Fact-retrieval and memory-update operator prompts (Tabs. 16–17).
AWM	Encoder unused at read() (full per-task-type workflow library prepended verbatim, no top- k).	Induction prompt (Tab. 18) from commit 8c0ff8cd; per-task-type pools (6 ALFWorld families); harness-native think:/act: formatting.
Memp	Encoder swapped to Qwen2.5-3B; cold-start build optional (defaults empty, grown via online reflect()); cosine ≥ 0.75 threshold (cosine-equivalent of upstream squared-L2 ≤ 0.5).	Workflow-build and failure-Adjustment prompts (Tabs. 19–20) from commit 3066a1b2.
Reflexion	Adapted to a <i>cross-task-type</i> reflection-transfer regime (failure reflections keyed by task_type, surfaced for new games of the same type), since our harness runs each game once rather than across multiple trials.	ReAct + reflection layering; reflector prompt (Tab. 21); two canonical ReAct exemplars layered alongside reflections.

Table 4: **Retrieval and generation hyperparameters per baseline.** “Budget” is the per-step LLM-call count for write/read/reflect. “ T ” is the sampling temperature.

Baseline	Settings
ReasoningBank	Budget 0/0/2; $T_{\text{action}}=0.0$, $T_{\text{rationale}}=0.1$, $T_{\text{extract}}=1.0$.
ExpeL	COMPARE_PAIRS_CAP=4, SUCCESS_CRITIQUE_NUM=8, MAX_NUM_RULES=10, NUM_FEWSHOTS=10; $T_{\text{insight}}=T_{\text{reflect}}=0.0$.
LightMem	LLMingua-2 compression $r=0.7$; stm_token_threshold=512; sensory Jaccard window = 3 at threshold 0.7; consolidation_every_n_episodes=10.
Mem0	Budget 2/0/1; both LLM calls at $T=0$.
AWM	Up to 5 most-recent successes per task type; $T_{\text{induce}}=0.1$, INDUCTION_MAX_TOKENS=2048.
Memp	Top- $K=10$; cosine threshold 0.75; deprecation rule hit ≥ 3 AND success/hit <0.5 ; $T_{\text{build}}=T_{\text{reflect}}=0.1$.
Reflexion	Reflect on failure only (1 call); 0 on success; max_reflections_per_type=5.

and ScienceWorld additionally set `--shuffle-tasks` to break the default `task_type`→`variation` clustering and load a frozen episode list via `--episodes-path episodes.jsonl`.

Table 5: Task pools used in online evaluation. ALFWorld and ScienceWorld sub-pools are produced by sampling on `task_type`; WebArena combines the shopping, shopping_admin, and gitlab task families.

Env	Task pool	Size	--max-agent-steps
ALFWorld	valid_seen + valid_unseen, 6 task families	274	50
ScienceWorld	HF test split, 7 task types sampled (see Appx. C.3)	660	100
WebArena	shopping + shopping_admin + gitlab task families	117	300

Models served. Open-weight roles (task agent, writer when Qwen, dreamer, embedder) are served as independent SGLang endpoints on the same evaluation node (see Hardware below); the Gemini-3-flash-preview task agent and Gemini-3.1-flash-lite-preview writer used on WebArena are accessed via the Google AI API. All endpoints expose the OpenAI-compatible `/v1/chat/completions` interface. Decoding uses temperature 0.7 and $\text{top-}p = 0.9$, with `enable_thinking = False` on Qwen3 endpoints.

Hardware. Training uses 8 NVIDIA H100 (80 GB) GPUs. Evaluation runs use one node with 4 NVIDIA GH200 (96 GB) GPUs serving all model endpoints (task agent, writer, dreamer, embedder).

Memory store and retrieval. Memory is persisted in LanceDB partitioned by `run_id`, with 2048-dim Qwen2.5-3B embeddings. At task start we issue a single retrieval against the task instruction and return up to $\text{top-}k = 3$ entries (hybrid kNN + salience rerank, 1500-token budget); during the episode we additionally refresh memory every 8 environment steps by re-querying with instruction || last-K actions || current observation and appending the top-1 retrieved entry to the user message. Retrieved entries are injected as raw `INSERT_*` blocks inside a `===Memory from past experience=== ... ===End Memory===` panel appended to the env-native system prompt—the same format used at training time, so no method gains from prompt-format mismatch.

Logging and metrics. For every task we record success, final environment score, episode length, retrieved entry IDs and token count, writer/dreamer events, and bank size, streamed to `per_task.jsonl`, `trajectories.jsonl`, and `dreamer_calls.jsonl`; the aggregate `summary.json` reports success rate, mean final score, end-of-run active and retired bank sizes, total wall-clock time, and per-role LLM call and token counts. Unless noted, we report success rate and mean final score over the full task stream and plot bank size and dreamer firings indexed by task position t to expose prequential learning dynamics rather than only end-of-stream aggregates.

B.3 Training Hyperparameters

Table 6 reports the full set of training hyperparameters for the ScienceWorld GRPO run, including model and optimization settings, rollout and generation parameters, environment and episode configuration, and reward shaping coefficients. The trained consolidator is applied to all three evaluation domains without further updates.

Table 6: Training hyperparameters for the ScienceWorld GRPO run.

Parameter	Value
<i>Model & optimization</i>	
Base model	Qwen3-14B
Algorithm	GRPO (agent loop)
Optimizer	AdamW
Learning rate	1×10^{-6}
LR schedule	constant
Warmup ratio	0.0
Entropy coefficient	0.0
KL loss coefficient (β)	1×10^{-3}
KL loss type	low-variance KL
Random seed	42
#GPUs	8
Tensor-parallel size	1
<i>Rollout & generation</i>	
Training steps	200
Batch size (prompts)	16
Rollout group size N	8
Sampling temperature (train)	1.0
Top- p (train)	0.9
Sampling temperature (val)	0.0
Top- p (val)	1.0
Max new tokens / turn	2,048
Max prompt tokens	3,072
Max response tokens	18,432
Max model length	21,504
GPU memory utilization	0.80
Thinking mode	disabled
<i>Environment & episode</i>	
Environment	ScienceWorld
Simplifications	teleport, openDoors, self-watering pots
Max env. steps / episode	50
Max assistant / user turns	40 / 40
History window (turns)	6
Valid-action sample size	20
Groups per prompt (k)	10
Evaluations per sample	12
Min. groups per task type	20
<i>Reward shaping</i>	
Format-penalty weight	0.5
MC-dropout weight	0.5
MC-dropout: min N	1
MC-dropout: sample fraction	0.25
MC-dropout: drop fraction	0.5
Task model	Qwen3.5-9B

C Artifact Details

C.1 Model License

Gemini-3.1-flash-lite-preview License: Proprietary

Gemini-3-flash-preview License: Proprietary

Qwen3-14B License: Apache 2.0

Qwen3.5-9B License: Apache 2.0

C.2 Software Versions

For our agentic RL environments, we build upon OpenTinker³, which provides unified task configurations across ALFWorld and ScienceWorld. Our training infrastructure is built on top of verl⁴.

C.3 Dataset Statistics

We report the dataset statistics for the three interactive benchmark environments used in our experiments: ALFWorld, ScienceWorld, and WebArena. ScienceWorld is used for both training-data construction and held-out evaluation; ALFWorld and WebArena are held-out only and not used during training. All environments’ statistics follow the data-processing and environment configurations of the original papers. Table 7 summarizes the train and test splits; Table 5 reports the evaluation pool.

Environment	# Task Types	# Train	# Test	Source
ALFWorld	6	3,553	274	Shridhar et al. [24]
ScienceWorld	30	3,604	1,802	Wang et al. [28]
WebArena	241	–	812	Zhou et al. [44]

Table 7: Train/test split statistics for the three interactive benchmark environments, using the original dataset releases. ALFWorld provides 3,553 training games and a held-out *seen + unseen* test set of 274 games across 6 compositional household task types [24]. ScienceWorld contains 30 task types with 7,207 parametric variations in total, split 50 % / 25 % / 25 % into train / dev / test sets [28]; we report train and test only. WebArena is held-out only and not used during training; it consists of 812 tasks instantiated from 241 intent templates across five self-hosted sites (Shopping, Shopping Admin/CMS, Reddit, GitLab, and Maps) [44].

WebArena. WebArena consists of long-horizon web-navigation tasks served via a self-hosted, sandboxed deployment. We use 117 held-out tasks sampled from three task families—shopping (e-commerce product search and checkout), shopping_admin (Magento admin operations), and gitlab (GitLab repository management). No WebArena trajectories are used for training; the consolidator trained on ScienceWorld is applied to WebArena without any additional updates, testing the cross-domain transfer claim of §5.2.

D Impact Statement

Auto-Dreamer is foundational research on long-term memory mechanisms for language agents. It does not introduce a new deployed system, a new dataset of human subjects, or a new generative capability tied to a specific application domain. All experiments are conducted in simulated agentic environments (ALFWorld, ScienceWorld, WebArena) that do not involve personal data or interaction with real users. As such, the immediate societal impact of this specific work is limited.

E Online Memory-Construction Prompts

This appendix lists the system prompts used by the three roles in our online memory pipeline: the per-trace *writer*, the cross-trace *auto-dreamer* synthesizer, and the environment *task agent*. We report verbatim text from `opentinker/memory_training/` and `opentinker/environment/` after stripping a small number of defensive phrases that we found contributed nothing to performance (these are clearly marked below).

Shared task-agent prompt across baselines. The task-agent prompt (Sec. E.3) is held *identical* across every baseline (`no_memory`, `writer_only`, `reflexion`, `expel`, `awm`, `memp`, `reasoningbank`, `mem0`, `lightmem`, `auto_dreamer`, `auto_dreamer_rl`). The only thing that differs across baselines is the contents of the `=== Memory from past experience ===` block injected at the end of the system prompt; the surrounding agent prompt is invariant. Memory-aware

³<https://github.com/open-tinker/OpenTinker>

⁴<https://github.com/volcengine/verl>

variants of the agent prompt (e.g. “treat memory as a hint, not a recipe”) were tested in ablations and found to be net-negative on the final aggregate score, so the reported runs use the un-instrumented agent prompt.

E.1 Writer Prompts

The writer reads ONE episode trace (marked SUCCESS or FAIL) and emits zero or more structured INSERT_SEMANTIC or INSERT_PROCEDURAL blocks. Output is parsed verbatim into the bank.

E.1.1 ALFWorld Writer

Success path.

You are a Memory Agent. Read an episode trace from a task agent and distill reusable knowledge into structured memory entries.

The task agent operates in ALFWorld -- a text-based household environment where it must complete tasks like "put a clean apple in the fridge" by issuing text commands (go to, take, clean, put, etc.).

You will receive ONE episode trace, marked SUCCESS or FAIL.

Your output is injected into a task agent's system prompt to help it succeed on NEW, unseen tasks of the same type.

OUTPUT FORMAT

You may emit one or more entries. Each entry must be one of:

INSERT_SEMANTIC

name: <short_id>
summary: <one-line description>
details: <full information>
END

INSERT_PROCEDURAL

name: <short_id>
type: <workflow|guide>
summary: <short description>
steps: ["step 1", "step 2", ...]
END

Nothing useful: NO_UPDATE

FORMAT RULES

- Use exact ALFWorld action names (go to, take, move, open, close, use, examine, heat, cool, clean, slice, inventory, look).
- Output entries then STOP.

Failure path.

[same header as Success]

A failed trace is useful for distilling the failure mode -- the specific unproductive action pattern observed.

INSERT_SEMANTIC

name: <short_id>
summary: <failure mode actually observed>
details: concrete description of the failure mode in this trace, quoting verbatim observations where possible.
END

If the trace is too short or noisy: NO_UPDATE

E.1.2 ScienceWorld Writer

Success path.

You are a Memory Agent. Read an episode trace and distill reusable knowledge into structured memory entries.

The task agent operates in ScienceWorld -- a text-based scientific reasoning environment with 30 distinct task types (measure-melting-point, test-conductivity, grow-plant, chemistry-mix, find-animal, mendelian-genetics, lifespan-*, inclined-plane-*, etc.). Each task unfolds over multiple rooms (kitchen, workshop, greenhouse, art studio, living room, bathroom, outside, foundry, bedroom, hallway).

The agent issues commands from templates such as:

teleport to ROOM	open OBJ	pick up OBJ
look at OBJ	look in OBJ	put down OBJ
move OBJ to OBJ	pour OBJ in OBJ	dunk OBJ in OBJ
mix OBJ	eat OBJ	use OBJ on OBJ
activate OBJ	deactivate OBJ	flush OBJ
connect OBJ to OBJ	disconnect OBJ	read OBJ
focus on OBJ	wait	wait1

Your output is injected into a task agent's system prompt to help it succeed on NEW, unseen variations of the SAME task type.

OUTPUT FORMAT

[INSERT_SEMANTIC / INSERT_PROCEDURAL blocks as in ALFWorld]

FORMAT RULES

- Use exact SciWorld action templates.
- Output entries then STOP.

E.1.3 WebArena Writer

Success path.

You are a Memory Agent. Read an episode trace from a web-browsing task agent and distill reusable knowledge into structured memory entries.

The task agent operates a real browser (Chromium, headless) via a high-level action API. Each turn it receives the page's accessibility tree (AXTree) as the observation and emits one action like ``click('123')``, ``fill('42', 'value')``, ``scroll(0, 200)``, ``keyboard_press('Enter')``, ``goto(url)``, or ``send_msg_to_user(text)``.

Your output is injected into a task agent's system prompt to help it succeed on NEW tasks involving similar widgets or workflows.

OUTPUT FORMAT

[INSERT_SEMANTIC / INSERT_PROCEDURAL blocks as in ALFWorld; semantic entries describe widget knowledge, procedural describe action sequences]

FORMAT RULES

- Refer to elements by their VISIBLE LABEL or ROLE as it appears in the AXTree, NOT by element bid numbers (bids change every page load).
- Use exact action names: `click`, `fill`, `scroll`, `keyboard_press`, `goto`, `select_option`, `hover`, `dblclick`, `drag_and_drop`, `send_msg_to_user`.
- Output entries then STOP.

Failure path. On WebArena we observed that behavioral failure entries (“submitted before verifying”, “didn’t wait for the page to load”) generalize net-negative; the failure prompt explicitly excludes them and only accepts concrete content/navigation mistakes:

A failed trace is useful ONLY when it shows a CONCRETE FACTUAL MISTAKE the agent made -- a wrong field value, wrong navigation target, wrong inferred number, wrong sub-page, wrong filter selection.

```

INSERT_SEMANTIC
name: <short_id>
summary: <one-line description of the specific content mistake>
details: which exact value / menu / page / number was wrong (e.g.
"agent picked Period 'Month' but goal asked for 'Year'", or
"agent navigated to Reports>Reviews when goal needed Reports>Bestsellers").
Quote the wrong action verbatim.
END

```

If the only failure-pattern is behavioural caution: NO_UPDATE

E.2 Auto-Dreamer Synthesizer Prompt

The same prompt is used across all environments.

You are a Memory Bank Synthesizer. Read a reference bank of memory entries from past task sessions and create a compact, high-quality output bank of synthesized entries. Only your synthesized entries will be shown to the task agent.

The task agent uses these memories to succeed on NEW, unseen tasks. Capture transferable knowledge -- patterns, procedures, and insights that generalize across task instances. Entry summaries are used as retrieval keys, so write clear, descriptive summaries.

Call ONE tool per turn. When satisfied, call `terminate`.

AVAILABLE TOOLS

```

Navigation (reference bank, read-only):
  search_memory(query, k=5)
  check_memory(ids=[...])    (up to 30 ids)
  get_source_trace(id)

```

Synthesis (output bank):

```
  synthesize(source_ids, type, name, summary, details?, steps?)
```

Control: terminate()

SYNTHESIS GUIDELINES

- Survey the reference bank broadly before synthesizing.
- Each entry should capture distinct, non-redundant knowledge.
- Look for patterns: shared procedures, recurring constraints, common strategies. Generalize when multiple entries support it.
- When entries disagree, resolve by frequency or note the conditions under which each applies; use get_source_trace to ground claims.
- Both procedural and semantic entries are valuable.
- Prefer actionable rules (priority "do X before Y", conditional "if Z, skip W") when well-supported.

GROUNDING RULES

- Every synthesized entry must cite source_ids drawn from the reference bank.
- Preserve concrete details that carry warning value: the exact forbidden command, the invalid action syntax the env rejected, the wrong object that ended an episode early.

E.3 Task-Agent Prompts (shared across all baselines)

The task agent is the policy that interacts with the environment. All baselines use the prompt below verbatim; the only difference between no_memory and the memory baselines is the presence of a == Memory from past experience == block (followed by the retrieved entries) appended to the system prompt at task start.

E.3.1 ALFWorld Task Agent

You are the Task Agent in an ALFWorld environment.
Your goal is to complete household tasks by executing actions.

CRITICAL RULE: Every observation includes "=== Available Actions ===".
You MUST pick EXACTLY ONE action from that list, word-for-word.
Any command not on the list will fail.

Memory (if provided below) describes high-level STRATEGIES in natural language. It is NOT a list of executable commands. Use it to decide WHICH action from the Available Actions list to pick, but always output a command copied verbatim from the list.

Output ONLY a single action command.

Example response: go to desk 1

E.3.2 ScienceWorld Task Agent

You are a Task Agent playing ScienceWorld -- a text-based scientific reasoning environment. Your goal is to complete the task described to you by issuing text commands, one per turn.

NAVIGATION -- USE TELEPORT. Rooms are connected by doors that may be closed. Instead of "go door to kitchen" (which often fails), always prefer:

teleport to kitchen / teleport to workshop / teleport to greenhouse
teleport to hallway / teleport to bedroom / teleport to art studio
teleport to living room / teleport to bathroom / teleport to outside
teleport to foundry

Teleport always succeeds and is always available.

ACTION TEMPLATES (any action emitted must match one):

teleport to ROOM	go OBJ	look around
look at OBJ	look in OBJ	inventory
open OBJ	close OBJ	pick up OBJ
put down OBJ	move OBJ to OBJ	pour OBJ in OBJ
dunk OBJ in OBJ	mix OBJ	eat OBJ
read OBJ	use OBJ on OBJ	
activate OBJ	deactivate OBJ	flush OBJ
connect OBJ to OBJ	disconnect OBJ	
focus on OBJ	wait	wait1

OBJECTS: each turn shows a "Visible objects" list -- those are the base names the env recognises right now. Compound names like "substance in metal pot" are also accepted. When an object is hidden inside a closed container, first `open` or `look in` to reveal it.

DISAMBIGUATION: when an action targets an object with multiple instances, the env replies "Ambiguous request: please enter the number..." -- emit the corresponding number on the next turn.

E.3.3 WebArena Task Agent

You are a web-browsing agent. Each turn, observe the AXTree and emit EXACTLY ONE action in a fenced block:

```
```action
click('123')
```
```

Use `bid` strings (e.g. [42]) from the AXTree to refer to elements.
Emit `send\_msg\_to\_user('answer')` when done.

Action space:

```
{action_desc} % auto-injected: 12 BrowserGym high-level actions
               % (click, fill, scroll, keyboard_press, goto,
               % select_option, hover, dblclick, drag_and_drop,
```



```
% noop, send_msg_to_user)
```

F Usage of LLMs

We used LLMs as a writing assistant to help us edit parts of the paper. Additionally, we utilize the power of CodeX and Claude Code to help us code faster. All AI-generated writing and code are manually checked and modified. There is no fully AI-generated content in the paper.

G Additional Per-Baseline Memory-Construction Prompts

Reproduced verbatim from `baselines/*/memory.py`, which re-read each baseline’s reference repo at the commit hash recorded in the file’s top docstring. Long verbatim few-shot examples are replaced by a `[...]` marker citing the precise file:line.

ReasoningBank. Successful-trajectory extraction (Tab. 8), failed-trajectory extraction (Tab. 9), and the memory-injection banner prepended to retrieved items at inference time (Tab. 10).

Table 8: **ReasoningBank — successful-trajectory extraction prompt**

You are an expert in household environment navigation. You will be given a user query, the corresponding trajectory that represents **how an agent successfully accomplished the task**.

Guidelines
You need to extract and summarize useful insights in the format of memory items based on the agent’s successful trajectory.
The goal of summarized memory items is to be helpful and generalizable for future similar tasks.

Important notes
- You must first think why the trajectory is successful, and then summarize the insights.
- You can extract *at most 3* memory items from the trajectory.
- You must not repeat similar or overlapping items.
- Prefer concrete, actionable procedures over abstract principles. Do not embed specific product names, queries, or literal string contents from the task.

Output Format
Your output must strictly follow the Markdown format shown below:

```
# Memory Item i
## Title <the title of the memory item>
## Description <one sentence summary describing when or when NOT to use the
memory item>
## Content <1-3 sentences describing the insights learned to successfully
accomplishing similar tasks in the future>
```

Table 9: **ReasoningBank - failed-trajectory extraction prompt**

You are an expert in household environment navigation. You will be given a user query, the corresponding trajectory that represents **how an agent attempted to resolve the task but failed**.

Guidelines
You need to extract and summarize useful insights in the format of memory items based on the agent’s failed trajectory.
The goal of summarized memory items is to be helpful and generalizable for future similar tasks.

Important notes
- You must first reflect and think why the trajectory failed, and then summarize what lessons you have learned or strategies to prevent the failure in the future.
- You can extract *at most 3* memory items from the trajectory.
- You must not repeat similar or overlapping items.
- Prefer concrete, actionable recovery procedures over abstract principles. Do not embed specific product names, queries, or literal string contents from the task.

Output Format (same Markdown schema as Table 8, with “successfully accomplishing” replaced by “avoid such failures and successfully accomplishing”).

Table 10: **ReasoningBank** — memory-injection banner prepended to retrieved memory items at inference time

Below are some memory items that I accumulated from past interaction from the environment that may be helpful to solve the task. You can use it when you feel it’s relevant. In each step, please first explicitly discuss if you want to use each memory item or not, and then take action.

ExpeL. A rule-operation grammar (Tab. 11) appended to the compare-critique template (Tab. 12, fired on each success/failure pair) and the all-success template (Tab. 13, fired every 8 successes).

Table 11: **ExpeL** — rule-operation format template

<OPERATION> <RULE NUMBER>: <RULE>

The available operations are: AGREE (if the existing rule is strongly relevant for the task), REMOVE (if one existing rule is contradictory or similar/duplicated to other existing rules), EDIT (if any existing rule is not general enough or can be enhanced, rewrite and improve it), ADD (add new rules that are very different from existing rules and relevant for other tasks). Each needs to CLOSELY follow their corresponding formatting below (any existing rule not edited, not agreed, nor removed is considered copied):

AGREE <EXISTING RULE NUMBER>: <EXISTING RULE>
 REMOVE <EXISTING RULE NUMBER>: <EXISTING RULE>
 EDIT <EXISTING RULE NUMBER>: <NEW MODIFIED RULE>
 ADD <NEW RULE NUMBER>: <NEW RULE>

Do not mention the trials in the rules because all the rules should be GENERALLY APPLICABLE. Each rule should be concise and easy to follow. Any operation can be used MULTIPLE times. Do at most 4 operations and each existing rule can only get a maximum of 1 operation.

Table 12: **ExpeL** — compare-critique prompt

{instruction}
 Here are the two previous trials to compare and critique:
TRIAL TASK: {task}
SUCCESSFUL TRIAL: {success_history}
FAILED TRIAL: {fail_history}
Here are the EXISTING RULES: {existing_rules}

By examining and contrasting to the successful trial, and the list of existing rules, you can perform the following operations: add, edit, remove, or agree so that the new list of rules is GENERAL and HIGH LEVEL critiques of the failed trial or proposed way of Thought so they can be used to avoid similar failures when encountered with different questions in the future. Have an emphasis on critiquing how to perform better Thought and Action. Follow the below format:
 [Rule-operation format from Table 11 appended verbatim.]

Table 13: **ExpeL** — all-success critique prompt

{instruction}
 Here are the trials: {success_history}
 Here are the EXISTING RULES: {existing_rules}

By examining the successful trials, and the list of existing rules, you can perform the following operations: add, edit, remove, or agree so that the new list of rules are general and high level insights of the successful trials or proposed way of Thought so they can be used as helpful tips to different tasks in the future. Have an emphasis on tips that help the agent perform better Thought and Action. Follow the below format:
 [Rule-operation format from Table 11 appended verbatim.]

LightMem. STM→LTM extraction (Tab. 14) and offline UPDATE/DELETE/IGNORE consolidation (Tab. 15).

Mem0. Fact extraction (Tab. 16) followed by the ADD/UPDATE/DELETE/NONE memory-update operator (Tab. 17).

Table 14: **LightMem — STM→LTM extraction prompt**

| |
|--|
| <p>You are a Personal Information Extractor.</p> <p>Your task is to extract all possible facts or information about the user from a conversation, where the dialogue is organized into topic segments separated by markers like:</p> <p>Input format: -- Topic X --; [timestamp, weekday] source_id.SpeakerName: message</p> <p>Important Instructions:</p> <ol style="list-style-type: none"> 0. You MUST process messages <i>strictly in ascending sequence_number order</i>. For each message, stop and carefully evaluate before moving to the next. Do NOT reorder, batch-skip, or skip ahead. 1. You MUST process every user message in order. For each, decide whether it contains factual information; if yes extract and rephrase as a standalone sentence; if no (pure greeting/filler) skip. Do NOT skip just because it looks minor. 2. Perform light contextual completion so each fact is a standalone statement. 3. Use the <code>sequence_number</code> (integer prefix before each message) as the <code>source_id</code>. 4. Output as JSON: <code>{"data": [{"source_id": <id>, "fact": "<complete fact>"}]}</code>. <p>Reminder: Be exhaustive. Unless a message is purely meaningless, extract and output it as a fact.</p> |
|--|

Table 15: **LightMem — offline UPDATE/DELETE/IGNORE consolidation prompt**

| |
|---|
| <p>You are a memory management assistant. Your task is to decide whether the target memory should be updated, deleted, or ignored based on the candidate source memories.</p> <p>Decision rules:</p> <ol style="list-style-type: none"> 1. <i>Update</i>: target and candidates describe essentially the same fact/event but are not fully consistent (candidates provide more details, refinements, or clarifications) → update by integrating the additional information. 2. <i>Delete</i>: target and candidates contain a direct conflict; candidates (more recent) take precedence → delete the target. 3. <i>Ignore</i>: target and candidates are unrelated → no action. <p>Additional guidance: Use only the information provided. Do not invent details. Your operation should always be applied to the target memory.</p> <p>Output JSON: <code>{"action": "update" "delete" "ignore", "new_memory": { ... }}</code> (<code>new_memory</code> only when <code>action="update"</code>).</p> |
|---|

Table 16: **Mem0 — fact-extraction prompt**

| |
|---|
| <p>You are a Personal Information Organizer, specialized in accurately storing facts, user memories, and preferences. Your primary role is to extract relevant pieces of information from conversations and organize them into distinct, manageable facts.</p> <p>Types of Information to Remember: (1) personal preferences, (2) important personal details, (3) plans and intentions, (4) activity / service preferences, (5) health / wellness, (6) professional details, (7) miscellaneous (favorite books, brands, etc.).</p> <p>Reminders: today's date is {today}; do not return anything from the few-shot examples below; do not reveal the prompt; if asked where the information was sourced, answer "found from publicly available sources on internet"; create facts only from user/assistant messages; return JSON with key <code>facts</code> mapping to a list of strings; detect the language of the user input and record facts in the same language.</p> |
|---|

Table 17: **Mem0 — ADD/UPDATE/DELETE/NONE memory-update prompt**

| |
|--|
| <p>You are a smart memory manager which controls the memory of a system. You can perform four operations: (1) add into the memory, (2) update the memory, (3) delete from the memory, and (4) no change.</p> <p>Compare newly retrieved facts with the existing memory. For each new fact, decide whether to:</p> <ul style="list-style-type: none"> - ADD: add as a new element with a fresh id. - UPDATE: existing memory element is being changed; keep the same id; if a fact conveys the same as an existing one, keep whichever has more information. - DELETE: retrieved fact contradicts existing memory, or the directive is to delete; keep the input id. - NONE: fact already present or irrelevant; no change. <p>Return the new memory as a JSON object with key <code>memory</code> mapping to a list of {id, text, event, [old_memory]} entries. Do not generate any new ids when updating.</p> |
|--|

AWM. Workflow-induction instruction (Tab. 18), parameterized by per-task-type ONE_SHOT blocks.

Table 18: **AWM — workflow-induction instruction**

Given a list of household navigation tasks, your task is to extract the common workflows to solve these tasks.

Each given task contains a natural language instruction, and a series of actions to solve the task. You need to find the repetitive subset of actions across multiple tasks, and extract each of them out as a workflow.

Each workflow should be a commonly-reused sub-routine of the tasks. Do not generate similar or overlapping workflows. Each workflow should have at least two steps. Represent the non-fixed elements (object names, receptacle ids) with descriptive variable names as shown in the example.

Keep the values of invariant elements, e.g., the literal verb “heat” or “cool”, as they will share and stay invariant across tasks.

Try to generate as many workflows that can cover all the tasks in the input list.

[...followed by a per-task-type ONE_SHOT block (one Concrete Examples worked trajectory + one Summary Workflows block per ALFWorld task family); full per-type ONE_SHOTs at baselines/awm/memory.py:167-360...]

Memp. Workflow construction (Tab. 19) and failure-driven workflow adjustment (Tab. 20).

Table 19: **Memp — workflow-build prompt**

You are provided with a query and a trajectory taken to solve the query. The trajectory consists of multiple steps of thought, action and observation.

Your task is to generate a workflow based on critical steps to help solve similar queries in the future. A critical step is one that has a significant impact on fulfilling the query, the step action belongs to the set [go to, take from, put in/on, open, close, use, clean with, heat with, cool with, examine, look], and the action’s outcome is successful and contributes positively to achieving the query.

Notice: Write the workflow as a natural, coherent paragraph (not as a bullet list or numbered steps). Use clear, concise language to describe what actions should be taken and in what general order.

—EXAMPLE WORKFLOW—

To solve this query, begin by identifying the most likely receptacles where the target object can be found and visit them one by one. After locating and taking the object, perform any required transformation such as cleaning at a sinkbasin, heating with a microwave, or cooling with a fridge. Finally, go to the destination receptacle and put the object in/on it to complete the task.

—EXAMPLE END—

Query: {query}
Trajectory: {trajectory}
 - **DO NOT** copy Thought:, Action:, or Observation: lines from the trajectory above.
 Output the workflow without any explanation or context:

Reflexion. Reflector prompt fired on failure only (Tab. 21).

Table 20: **Memp** — failure-Adjustment prompt

You are a helpful assistant. You are given a workflow, a reward, and a trajectory.
 Reward is a number between 0 and 1; 1 means the trajectory is successful, 0 means failed.
 If the reward is False (i.e., the trajectory guided by the workflow did not successfully complete the task), then analyze why the task was not completed based on the trajectory and the workflow.
 After that, refine the workflow to make it more accurate and robust, so that it can better guide the completion of the task.
Workflow: {workflow}
Reward: {reward}
Trajectory: {trajectory}

[...six verbatim per-task-type Output Example Workflows (one each for pick_and_place, pick_clean_then_place, pick_heat_then_place, pick_cool_then_place, look_at_obj, pick_two_obj) omitted; full prompt at baselines/memp/memory.py:121-127...]
 Keep your output in the format below:
 <Analysis> your analysis here </Analysis>
 <Workflow> your adjusted workflow here </Workflow>

Table 21: **Reflexion** — failure-reflection prompt

You are a self-reflection agent. The agent attempted the following task and FAILED.
Task: {task}
Trajectory: {traj}

Write a short paragraph (at most 4 sentences) reflecting on what went wrong and a concrete strategy to try next time on a SIMILAR task. Be specific and actionable.
 Reflection:

H Bootstrap Confidence Intervals

We complement the point estimates in Table 1 with bootstrap 95% confidence intervals on per-method success rate. For each domain we resample tasks with replacement ($N_B = 10,000$) and report the bootstrap mean and 95% percentile interval. WebArena CIs are computed on the per-task-family macro average to match the main-text metric.

Table 22: Bootstrap 95% CIs on continual-memory deployment success rate ($N_B = 10,000$). ScienceWorld and ALFWorld are evaluated with per-task SR; WebArena uses per-task-family macro-averaged SR over 117 tasks across shopping, shopping_admin, and gitlab.

| Method | ScienceWorld | ALFWorld | WebArena |
|---------------------|-----------------------------|-----------------------------|--------------------------|
| No memory | 28.69 [25.18, 32.14] | 30.83 [26.40, 35.31] | 44.6 [36.0, 53.2] |
| Reflexion | 29.55 [26.07, 33.10] | 49.17 [43.59, 54.73] | 46.4 [37.7, 54.9] |
| ExpeL | 28.33 [24.97, 31.76] | 33.59 [28.79, 38.48] | 50.8 [41.8, 59.7] |
| AWM | 30.18 [26.61, 33.72] | 32.75 [28.16, 37.47] | 52.0 [43.1, 60.8] |
| Memp | 30.36 [26.90, 33.96] | 31.40 [27.09, 35.76] | 50.8 [41.7, 59.7] |
| ReasoningBank | 30.86 [27.32, 34.55] | 31.11 [26.56, 35.84] | 49.8 [41.1, 58.6] |
| Mem0 | 26.79 [23.39, 30.21] | 30.61 [25.81, 35.42] | 50.3 [41.5, 59.2] |
| LightMem | 28.08 [23.90, 32.37] | 31.16 [26.72, 35.53] | 52.0 [43.1, 60.6] |
| Mem- α | 29.97 [26.43, 33.51] | 57.40 [52.04, 62.59] | — |
| UMEM | 34.07 [30.51, 37.71] | 58.43 [53.15, 63.65] | — |
| Auto-Dreamer | 41.07 [37.53, 44.70] | 60.21 [54.65, 65.59] | 52.3 [43.5, 60.9] |

I Case Study: LightMem vs Auto-Dreamer

We include a qualitative case study to illustrate how Auto-Dreamer achieves compactness without sacrificing task success. We run Auto-Dreamer and LightMem on the same 96 episodes from the ScienceWorld Lifespan-compare category, using the same task order, task agent, writer, retriever, and memory-token budget. Both methods solve 48 out of 96 tasks, yielding identical success rate of

50.0%. The difference is therefore not task accuracy in this slice, but the structure and size of the memory bank that supports future retrieval.

At episode 90, LightMem has 265 active entries totaling 17,512 tokens, whereas Auto-Dreamer has 14 active entries totaling 716 tokens. Thus, in this run, Auto-Dreamer maintains a $24.5\times$ smaller active bank in tokens and an $18.9\times$ smaller bank in entry count while matching LightMem’s task success.

Table 23: Case study on the ScienceWorld `lifespan-compare` category. Auto-Dreamer matches LightMem’s success while maintaining a substantially smaller active memory bank.

| Method | Success | Active entries at $t=90$ | Active tokens at $t=90$ |
|--------------|---------------|--------------------------|-------------------------|
| LightMem | 48/96 (50.0%) | 265 | 17,512 |
| Auto-Dreamer | 48/96 (50.0%) | 14 | 716 |

LightMem accumulates surface-level duplicates. LightMem’s bank is dominated by near-verbatim restatements of task instructions and local state observations. Among the 265 active entries at $t=90$, 49 are paraphrases of the task instruction. The first four such entries are byte-identical:

“The task is to find the animal with the longest life span, then the shortest life span. The animals are located in the ‘outside’ location. Additionally, there are sequential subgoals to focus on the animal with the shortest life span.”

The bank also contains repeated trivial state shards, including four byte-identical copies of “*The agent’s inventory contains an orange.*” and two copies of “*The agent has taken 0 moves so far.*” It further stores multiple reorderings of the same room description:

[ltn#32]: “In the foundry, there is a blast furnace that is turned off and has a closed door, a sink that is turned off and contains nothing, a table that contains nothing, and a door to the outside that is open.”

[ltn#33]: “In the foundry, there is a sink that is turned off and contains nothing, a blast furnace that is turned off and has a closed door, a table that contains nothing, and a door to the outside that is open.”

[ltn#34, #35]: further reorderings of the same four objects.

The remaining bank includes many long summaries that recapitulate the full visited world state together with histories of invalid actions. In this run, LightMem’s consolidation step fires nine times but retires no active entries, so memory grows monotonically.

Auto-Dreamer replaces instances with abstractions. Auto-Dreamer’s first consolidation trigger fires at episode 9. At that point, the writer has emitted candidate memories from five trajectories, each tied to a different concrete focus target: *crocodile*, *egg giant tortoise*, *baby brown bear*, *baby elephant*, and a generic *animal*. Rather than preserving these as separate task-specific entries, Auto-Dreamer collapses them into a single procedural rule:

General procedure for lifespan comparison tasks:

- teleport to outside
- focus on animal (*prefer adult over juvenile or egg*)

This entry preserves the reusable structure shared across the five trajectories while omitting episode-specific target names that would compete at retrieval time.

Auto-Dreamer also synthesizes memories that abstract recurring failure modes. For example, from failed trajectories in which the agent focused on *baby baby beaver* and *chameleon egg*, it writes:

Common incorrect targets in lifespan comparison tasks: *Focusing on juveniles or eggs instead of adult animals can lead to failure in lifespan comparison tasks.*

A later consolidation event, using five additional failed trajectories from episodes 10–17, generalizes the same pattern into a higher-level procedural memory about focus-target accuracy. Together, the retained entries cover three complementary facts: where relevant animals are located, which targets are usually incorrect, and how to choose the correct adult target.

Retrieval becomes less redundant. The effect is visible at evaluation time. On `lifespan-shortest-lived::119` at episode 90, both methods succeed. However, the top retrieved memories differ sharply. LightMem retrieves the same task-instruction sentence three times, each from a different timestamp. Auto-Dreamer retrieves three distinct pieces of information: the relevant location, a common anti-pattern, and the success criterion for selecting the target. Thus, even when both agents solve the task, Auto-Dreamer uses each retrieval slot to expose a different abstraction, whereas LightMem spends multiple slots on duplicated content.

Why this case matters. This example illustrates the mechanism behind Auto-Dreamer’s memory-efficiency gains. LightMem’s prompted consolidation is conservative: it tolerates near-duplicates that differ only in surface form and does not reliably merge repeated observations into higher-level rules. Auto-Dreamer instead treats consolidation as region rewriting: it retires multiple concrete memories and replaces them with fewer provenance-grounded abstractions. In this case, LightMem grows approximately linearly with episodes, reaching 265 active entries by episode 90, whereas Auto-Dreamer reaches a compact steady state of roughly 10–14 active entries within the first half of the stream.

Overall, on `lifespan-compare`, both systems achieve the same 50.0% success rate over 96 episodes, but Auto-Dreamer maintains a $24.5\times$ smaller active memory bank in tokens. This supports the main quantitative finding that learned offline consolidation improves the success–cost tradeoff not by merely deleting memories, but by replacing redundant local observations with compact, reusable abstractions.

J Case Study: AWM vs Auto-Dreamer

We include a second case study to separate memory compactness from memory usefulness. We compare AWM and Auto-Dreamer on the ScienceWorld `find-entity` category under the same online evaluation stream. Both methods use the same task agent, writer, retriever, prompt format, and task order; they differ only in the long-horizon memory that is retained and consolidated. AWM induces a compact library of natural-language workflow templates from successful trajectories, whereas Auto-Dreamer rewrites writer-emitted memories into synthesized semantic and procedural lessons.

After 80 capped episodes, the two methods have nearly identical active-bank footprints: AWM stores 10 entries totaling 816 tokens, while Auto-Dreamer stores 13 entries totaling 795 tokens. The performance gap is nevertheless large: Auto-Dreamer solves 58 out of 80 tasks (72.5%), compared with 21 out of 80 for AWM (26.2%). Successful Auto-Dreamer episodes are also shorter, averaging 7.4 steps compared with 20.0 steps for AWM. Thus, in this category, the advantage is not explained by a larger memory bank, but by what the bank contains.

Table 24: Case study on ScienceWorld `find-entity`. AWM and Auto-Dreamer end the 80-episode stream with nearly identical active memory footprints, but Auto-Dreamer achieves substantially higher success and shorter successful trajectories.

| Method | Success | Avg. steps on successes | Active entries at $t=80$ | Active tokens at $t=80$ |
|--------------|---------------|-------------------------|--------------------------|-------------------------|
| AWM | 21/80 (26.2%) | 20.0 | 10 | 816 |
| Auto-Dreamer | 58/80 (72.5%) | 7.4 | 13 | 795 |

AWM stores compact but underspecified workflows. At episode 80, AWM’s memory consists of a single `find-entity` workflow bucket with 10 entries. These entries are concise action templates, such as:

Workflow: Locate {object} by scanning likely receptacles
think: An {object} is more likely to appear in {receptacle_list}. Check candidates one by one until found.
act: go to {receptacle_1}
act: open {receptacle_1}
act: go to {receptacle_2}
act: open {receptacle_2}

and:

Workflow: Focus and pick up {object}
think: The {object} is visible and can be picked up.
act: focus on {object}
act: pick up {object}

The bank also contains several near-paraphrases of a canonical recipe:

focus on {object} → pick up {object} → teleport to {target_location} → move {object} to {target_receptacle}.

These workflows are compact, but they primarily encode how to act once the target has already been identified. They do not encode the key semantic precondition for `find-entity`: the visible objects in the spawn room are often distractors, and the target entity is frequently elsewhere. In particular, the AWM bank contains no negative rule saying not to focus on salient but incorrect objects such as a banana, orange, apple, refrigerator, or bee hive before identifying the requested entity.

Auto-Dreamer stores identification knowledge and negative lessons. Auto-Dreamer’s bank at episode 80 has 13 entries: 10 synthesized by the dreamer and 3 fresh writer entries that have not yet been consolidated. Its highest-leverage entries include both procedural and semantic abstractions. A representative synthesized procedural entry is:

find-entity-general-procedure-synthesized
- *teleport to outside area*
- *focus on the entity*
- *pick up the entity*
- *teleport to the destination room*
- *move the entity to the specified container*

This entry is synthesized from five retired writer notes, with provenance tracing back to 22 originating trajectories. Its predecessors include concrete variants such as moving an entity to a yellow box, a blue box, or another destination container. Auto-Dreamer therefore collapses many task-specific action traces into one reusable procedure.

More importantly, Auto-Dreamer also stores a negative semantic lesson that AWM’s success-only workflow induction does not produce:

avoid-wrong-focus
The agent should avoid focusing on objects that are not the target entity before completing the task setup. Focusing on incorrect objects, such as a bee hive, orange, or apple, leads to early failure. It is crucial to identify the correct target entity first before performing any interaction actions.

This short entry is synthesized from failed trajectories and remains active through episode 80. The resulting bank covers four complementary facets of the task: where to search first, what objects to avoid, the canonical pick-and-move procedure, and common destination containers.

A side-by-side episode. Consider task `find-animal:254` at episode 29:

“Your task is to find a(n) animal. First, focus on the thing. Then, move it to the blue box in the living room.”

Both agents start in the same kitchen. The opening observation includes a counter containing a bowl with a red apple, a banana, an orange, and a potato, along with closed storage objects such as a cupboard, freezer, and fridge.

The AWM agent receives its retrieved workflow templates and immediately executes:

focus on banana

The episode terminates after one step with final score -1.0 . The failure is consistent with the contents of the AWM bank: several workflows begin with `focus on {object}`, but none encode that a banana is not an animal or that the agent should first leave the misleading spawn location.

The Auto-Dreamer agent retrieves three compact entries, including facts that the entity is typically outside and should ultimately be moved to the living room. It then follows the successful trajectory:

```
teleport to outside  
focus on common toad  
pick up common toad  
teleport to living room  
move common toad to blue box
```

The episode completes successfully in five steps. The task, starting observation, task agent, and writer are the same; the difference is the long-horizon memory exposed to the agent.

Why AWM struggles on find-entity. AWM is well matched to settings where successful demonstrations share a stable action skeleton. However, *find-entity* requires both procedural knowledge and identification knowledge. The agent must decide what counts as the requested entity, avoid salient distractors, and often navigate away from the initial room before interacting. A compact action template cannot express these preconditions unless they appear explicitly in the induced workflow.

The logs support this interpretation. Across the 80 capped tasks, AWM’s first action is a *focus on . . .* command in 20 episodes, and 11 of those episodes terminate immediately with score -1.0 after focusing on an incorrect object. These failures do not enter AWM’s success-derived workflow pool. Auto-Dreamer, in contrast, can consolidate writer notes from failed episodes and synthesize negative lessons such as *avoid-wrong-focus*. This gives the agent a compact rule about what not to do, rather than only a recipe for what to do after the correct target has already been found.

Summary. On ScienceWorld *find-entity*, Auto-Dreamer reaches 72.5% success with 7.4 steps per successful episode, while AWM reaches 26.2% success with 20.0 steps per successful episode. The two methods end the run with nearly identical active-bank footprints: 13 entries and 795 tokens for Auto-Dreamer versus 10 entries and 816 tokens for AWM. This case shows that Auto-Dreamer’s gains are not merely a consequence of compactness; they come from consolidating the right abstractions, including negative lessons from failure trajectories that success-only workflow induction fails to capture.